Dossier de projet

TaskFlow Manager

API REST Symfony et application web React

Titre professionnel Concepteur Développeur d'Applications

David Ongaro

Formation suivie à Global Digital University

Session 2026 - 03/10/2026

Sommaire

1. Besoin du projet....4
2. Présentation du projet....4
2.1 Contexte....4
2.2 Objectifs....4
3. Cahier des charges....4
3.1 Les utilisateurs....4
3.2 Les fonctionnalités....4
3.3 Les contraintes....5
4. Gestion de projet....5
4.1 Méthode....5
4.2 Git et GitHub....6
5. Spécifications fonctionnelles....7
5.1 Cas d'utilisation....7
5.2 Matrice d'autorisation....7
5.3 Maquettes....8
6. Spécifications techniques....9
6.1 Technologies....9
6.2 Environnement de travail....9
6.3 Architecture générale....9
6.4 Architecture de l'API....9
6.5 Architecture du front....10
7. Base de données....10
7.1 Modèle....10
7.2 Mise en place....11
8. Réalisations back-end....11
8.1 Les règles métier....11
8.2 L'accès aux données....12
8.3 La sécurité de l'API....13
9. Réalisations front-end....14
9.1 Le client HTTP....14
9.2 Les routes protégées....14
9.3 Une interface qui s'adapte aux droits....15
9.4 Les écrans....16
10. Sécurité....17
11. Tests....18
11.1 Stratégie....18
11.2 Jeu de données : changement d'état d'une tâche....18
12. Déploiement et DevOps....19
12.1 Architecture de déploiement....19
12.2 Docker....19
12.3 Intégration continue....20
12.4 Mise en production....20
13. Bilan....21
14. Annexes....21
14.1 Cas d'utilisation par rôle....21
14.2 Autres maquettes....23

14.3 Pipeline CI du front.....25
14.4 Liens.....25

1. Besoin du projet

La société TaskFlow gérait ses projets et ses tâches avec une API, utilisable uniquement par requêtes HTTP. Ce n'était pas pratique au quotidien et pas accessible à tous les collaborateurs. J'ai donc réalisé TaskFlow Manager, composé de deux applications : une **API REST en Symfony 7** (PHP 8.2, Doctrine, MySQL, authentification JWT) et une **application web en React 19** (Vite, Material UI) qui consomme cette API.

L'application permet de gérer des projets, leurs membres, leurs tâches et leurs tags, avec des droits différents selon le rôle (manager, owner, membre, assigné). Toutes les règles d'accès sont vérifiées par l'API. Le projet est testé (110 tests côté API, 62 côté front), conteneurisé avec Docker, vérifié par une intégration continue GitHub Actions et déployé sur Render.

2. Présentation du projet

2.1 Contexte

TaskFlow utilise une API pour suivre ses projets. Sans interface, chaque action (créer un projet, assigner une tâche…) demandait d'écrire une requête HTTP à la main. L'entreprise a demandé une interface web pour gérer ses projets simplement.

2.2 Objectifs

- Fournir une API sécurisée, documentée et testée ;
- Proposer une interface web simple, qui s'adapte aux droits de chaque utilisateur ;
- Pouvoir installer et déployer le projet facilement (Docker, CI, hébergement en ligne).

3. Cahier des charges

3.1 Les utilisateurs

Rôle	Description
Manager	Administre les utilisateurs, a tous les droits sur les projets
Owner	Créateur d'un projet : il le gère (modification, archivage, membres, tâches, tags)
Membre	Consulte le projet auquel il participe, ses tâches et ses tags
Utilisateur assigné	Peut faire évoluer l'état de la tâche qui lui est assignée
Utilisateur	Utilisateur connecté, qui peut créer ses propres projets

3.2 Les fonctionnalités

- Connexion et déconnexion, consultation et modification de son profil ;
- Projets : liste, détail, création, modification, archivage et restauration, gestion des membres ;
- Tâches : création, modification, suppression, assignation, état, priorité, échéance, filtres et tri ;
- Tags : création, renommage, suppression, association aux tâches ;
- Administration des utilisateurs (managers uniquement) : création, modification, désactivation, suppression.

3.3 Les contraintes

- Sécurité : chaque utilisateur ne voit et ne modifie que ce qui le concerne. Les droits sont vérifiés par l'API ;
- Un projet terminé ou annulé est archivé : il reste consultable mais n'est plus modifiable ;
- Le front n'a pas de base de données : toutes les données passent par l'API.

4. Gestion de projet

4.1 Méthode

J'ai travaillé en **Kanban** avec Trello (colonnes Backlog, To Do, In Progress, Merged). Les cartes sont classées par étiquette (Connexion, User, Project, Task, Tag, Tech). Chaque carte est rédigée comme une **user story**, suivie d'une partie technique qui sert de critères d'acceptation (route, rôles autorisés…).

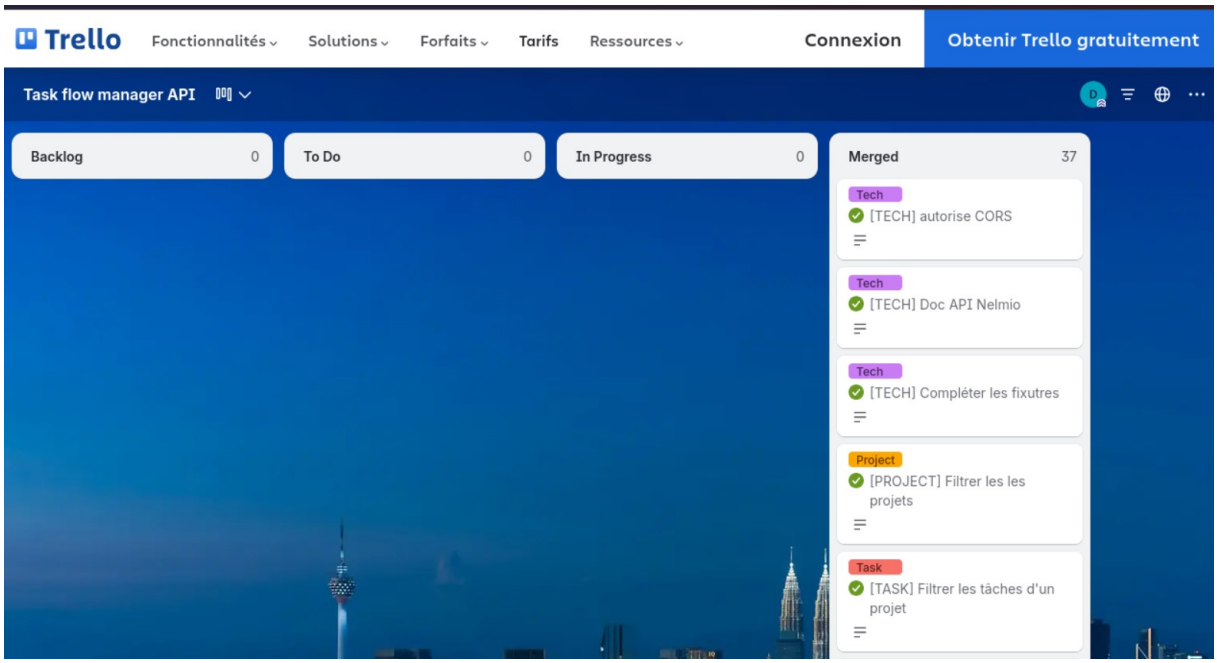
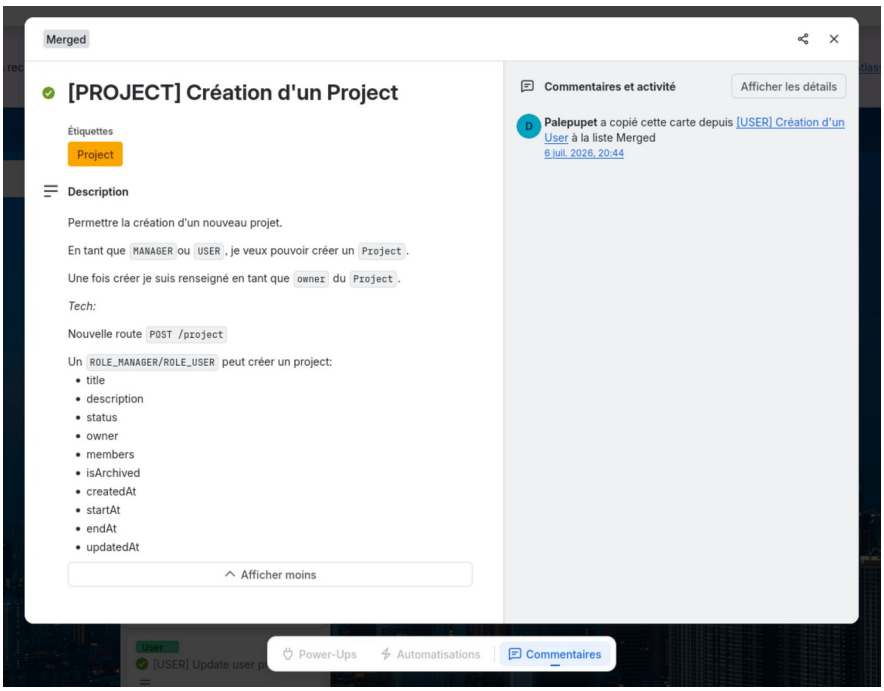


Tableau Trello du projet



Exemple de carte : user story et partie technique

4.2 Git et GitHub

Chaque carte donne une branche (feat/, fix/, tech/, refacto/), puis une **pull request** vérifiée par la CI avant d'être fusionnée dans main.

```
d41e299 Merge pull request #31 from palepupet/feat/add-tag-into-task
7feeb12 (origin/feat/add-tag-into-task, feat/add-tag-into-task) Feature - associer un tag a une task
afc9cb5 Merge pull request #30 from palepupet/feat/delete-tag
0b002ea (origin/feat/delete-tag, feat/delete-tag) Feature - supprimer un tags
92fc7d0 Merge pull request #29 from palepupet/feat/patch-tag
496bcac (origin/feat/patch-tag, feat/patch-tag) Feature - modifier les tags
1f48667 Merge pull request #28 from palepupet/feat/get-tags
5d0b187 (origin/feat/get-tags, feat/get-tags) Feature - get all tags
2ffdba8 Merge pull request #27 from palepupet/feat/create-tag
3e16486 (origin/feat/create-tag, feat/create-tag) Feature - creation Tag + post
00d15aa Merge pull request #26 from palepupet/tech/refacto-and-readme-completion
25f6e8f (origin/tech/refacto-and-readme-completion, tech/refacto-and-readme-completion) Tech - refacto et completion Readme
fd21742 Merge pull request #25 from palepupet/refacto/add-missing-test
532a7d9 (origin/refacto/add-missing-test, refacto/add-missing-test) Refacto - refacto, update and add missing test
33b96a9 Merge pull request #24 from palepupet/refacto/change-assigneeId-to-assignee
040b9b5 (origin/refacto/change-assigneeId-to-assignee, refacto/change-assigneeId-to-assignee) Refacto - change assigneeId to assignee
1a0db59 Merge pull request #23 from palepupet/feat/update-task-status
9c90b2f (origin/feat/update-task-status, feat/update-task-status) Feature - utilisateur assignee, tache open/close possible
0ba622B Feature - suppression tache
f7708d7 Merge pull request #22 from palepupet/feat/detail-update-task
460c365 (origin/feat/detail-update-task, feat/detail-update-task) Feature - tâche spécifique + modification
3ab846b Merge pull request #21 from palepupet/feat/create-task
59a8eb8 (origin/feat/create-task, feat/create-task) Feature - ajout Task
5820749 Merge pull request #20 from palepupet/tech/refacto-add-remove-member
844082b (origin/tech/refacto-add-remove-member, tech/refacto-add-remove-member) Tech - refacto ajout, suppression members
7ac3427 Merge pull request #19 from palepupet/feat/only-read-archived-project
2e32b57 (origin/feat/only-read-archived-project, feat/only-read-archived-project) Feature - ajout, suppression de membres
f9661f6 Feature - projet archivé en lecture uniquement
9026e2b Merge pull request #18 from palepupet/feat/archive-restore-project
a4ec8db (origin/feat/archive-restore-project, feat/archive-restore-project) Feature - restaure, archive project
b31f59a Merge pull request #17 from palepupet/feat/update-project
a3ac2fe (origin/feat/update-project, feat/update-project) Feature - update project
c3cbcfd Merge pull request #16 from palepupet/feat/list-specific-project
cd71ffe (origin/feat/list-specific-project, feat/list-specific-project) Feature - liste project specifique
b6a7489 Merge pull request #15 from palepupet/feature/get_projects
3af3cd4 (origin/feature/get_projects, feature/get_projects) Feature - liste tous les projects
166f1bc Merge pull request #14 from palepupet/tech/fixture-project
4a2852c (origin/tech/fixture-project, tech/fixture-project) Tech - creation fixture project
e5de27a Merge pull request #13 from palepupet/feat/create-project
2bea40e (origin/feat/create-project, feat/create-project) Feature - creation d'un project
9e7d475 Merge pull request #12 from palepupet/tech/user_fixture
7c73006 (origin/tech/user_fixture, tech/user_fixture) Tech - fixture user, manager
a16ed6a Merge pull request #11 from palepupet/tech/refacto_user
0fdc7da (origin/tech/refacto_user, tech/refacto_user) Tech - refacto
439f7e8 Merge pull request #10 from palepupet/feat/delete-user
d65587a (origin/feat/delete-user, feat/delete-user) Feature - delete user
fc71899 Merge pull request #9 from palepupet/feature/update-specific-user
659bba9 (origin/feature/update-specific-user, feature/update-specific-user) Feature - update user specifique
2fa08e1 Merge pull request #8 from palepupet/feat/user-specific
404645d (origin/feat/user-specific, feat/user-specific) Feature - information user specifique
637b3c8 Merge pull request #7 from palepupet/feat/get_all_users
ab2d9b6 (origin/feat/get_all_users, feat/get_all_users) Feature - listing des users
fe458f9 Merge pull request #6 from palepupet/feat/create_user
28396cd (origin/feat/create_user, feat/create_user) Feature - creation d'un user
```

Historique des fusions (git log)

5. Spécifications fonctionnelles

5.1 Cas d'utilisation

J'ai réalisé un diagramme de cas d'utilisation par rôle. Celui du manager est présenté ici, les autres sont en annexe.

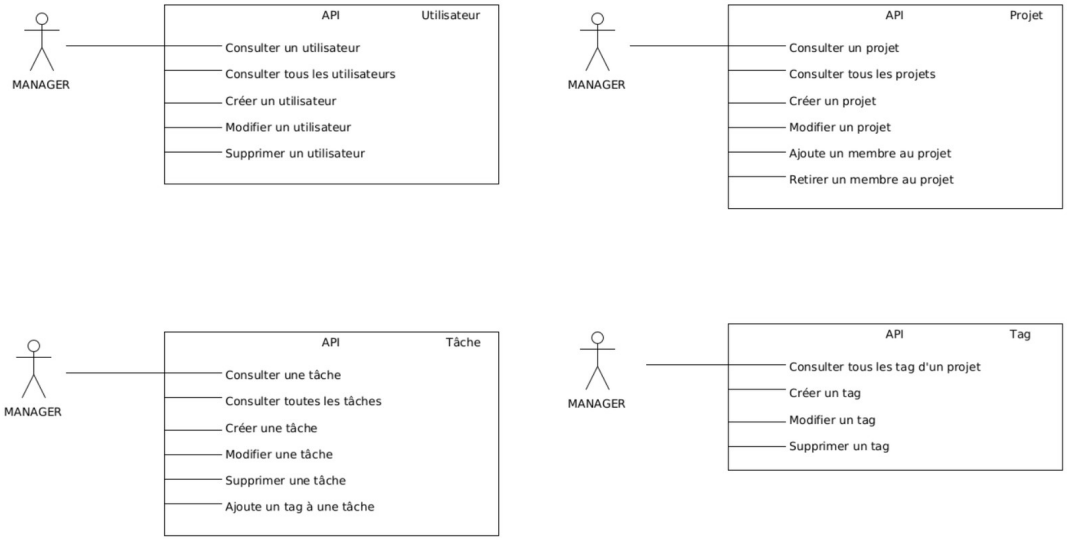


Diagramme de cas d'utilisation - Manager

5.2 Matrice d'autorisation

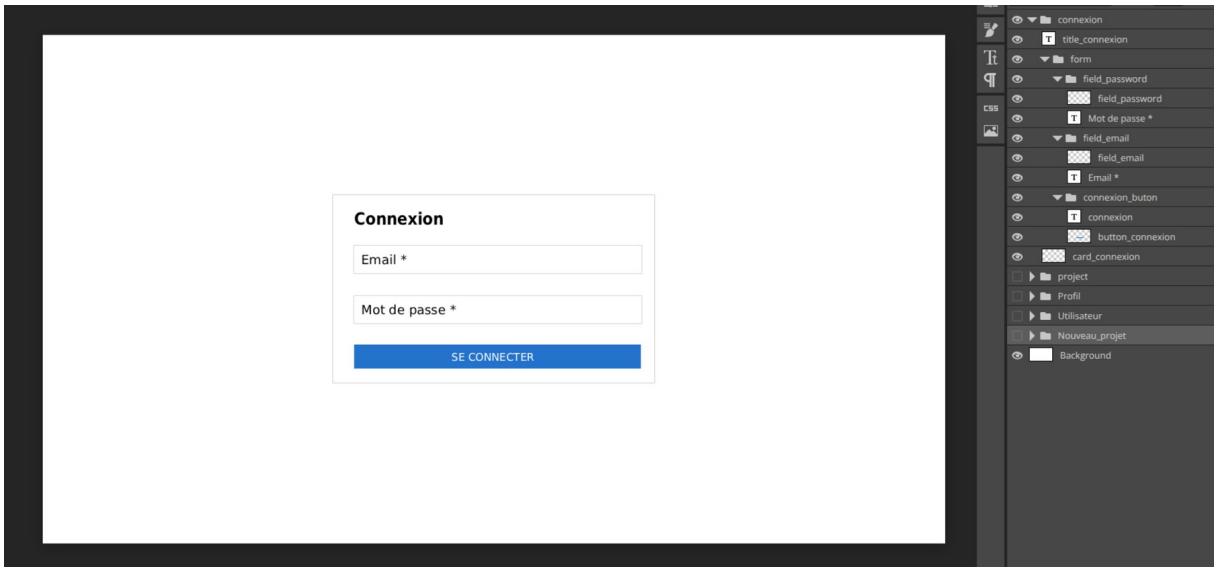
La matrice croise chaque action avec chaque rôle (vert : autorisé, rouge : refusé). Elle a servi de référence pour coder les contrôles d'accès de l'API, pour afficher ou masquer les boutons du front et pour écrire les tests.

Action	MANAGER	OWNER	MEMBRE	ASSIGNÉ TÂCHE	UTILISATEUR	HORS PROJET
Auth						
POST /auth/login						
GET /me						
PATCH /me						
Utilisateurs						
Lister / voir / créer / modifier /supprimer un user						
Projets (en cours)						
Créer un projet						
Lister les projets						
Voir un projet spécifique						
Modifier / archiver / restaurer						
Ajouter / retirer des membres						
Projets (archivés)						
Lecture (projet, tâches, tags)						
Écriture						
Tâches						
Lister / voir celles du projet						
Créer une tâche						
Modifier : titre, description, Priorité, échéance, assignee						
Supprimer une tâche						
Passer l'état à en cours						
Passer l'état à ouvert / terminé						
Associer un tag à une tâche						
Tags						
Lister les tags						
Créer / renommer / supprimer un tag						

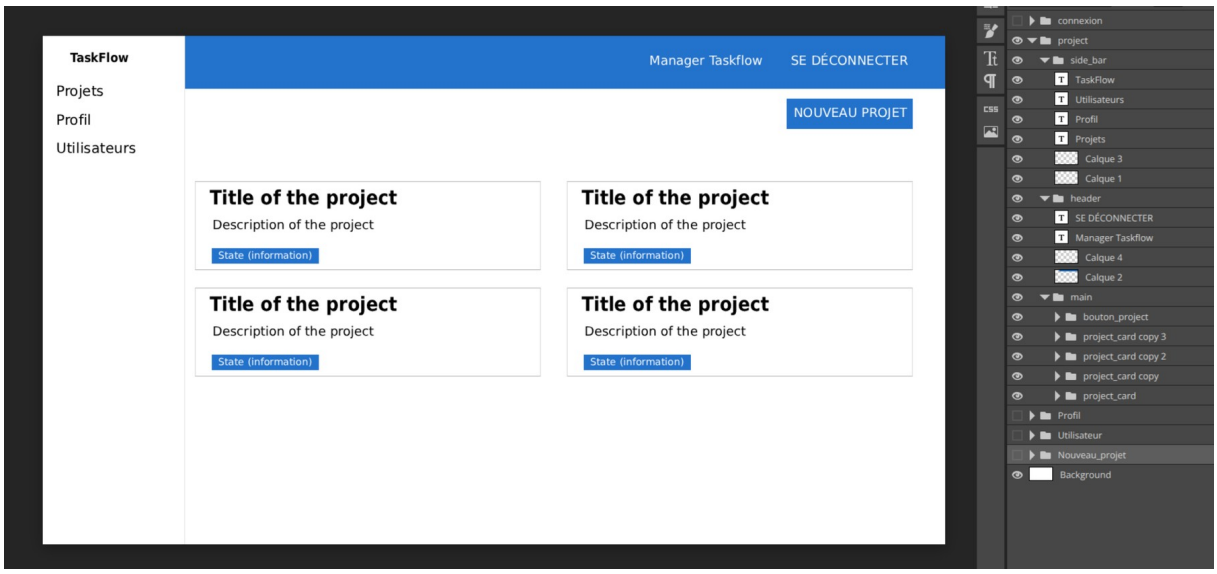
Matrice d'autorisation

5.3 Maquettes

Les maquettes ont été réalisées avec Photopea. Les écrans connectés partagent la même mise en page : une barre latérale (Projets, Profil, Utilisateurs), un bandeau avec le bouton de déconnexion et une zone de contenu.



Maquette - Connexion



Maquette - Liste des projets

6. Spécifications techniques

6.1 Technologies

	API	**Application web**
Langage	PHP 8.2	JavaScript
Framework	Symfony 7.2	React 19, Vite
Données	Doctrine ORM, MySQL 8 (SQLite pour les tests)	Pas de base : appels à l'API
Sécurité	Security, LexikJWT, NelmioCors	Token JWT, routes protégées
Interface	Documentation Swagger (NelmioApiDoc)	Material UI, React Router
Qualité	PHPStan, PHPMD, PHP CS Fixer, PHPUnit	ESLint, Vitest, Testing Library
Déploiement	Docker (php:8.2-apache), Render	Docker (Node + nginx), Render

6.2 Environnement de travail

PhpStorm (plugin Symfony), Composer, Node.js et npm, Docker et Docker Compose, Git et GitHub, Bruno pour tester les routes, Trello, sur un poste Linux (Zorin OS).

6.3 Architecture générale

Le front et l'API sont deux applications indépendantes, chacune avec son dépôt, sa CI et son déploiement. Elles communiquent en HTTP/JSON ; le front envoie le token JWT dans le header Authorization: Bearer.

6.4 Architecture de l'API

L'API est organisée en couches :

Couche	**Rôle**
Controller	Reçoit la requête, appelle les autres couches, renvoie la réponse JSON
Dto	Données en entrée (avec validation) et en sortie (ce qui est exposé)

Entity	Données et règles métier (droits, cycle de vie)
Enum	Valeurs limitées : statut, état, priorité, rôle
Repository (+ interfaces)	Requêtes vers la base avec Doctrine
Security	Authentification JWT, refus des comptes désactivés
Http	Lecture du JSON, format unique des erreurs

Parcours d'une requête : pare-feu JWT -> contrôleur -> règle métier de l'entité -> DTO et validation -> règles de cohérence -> enregistrement Doctrine -> réponse JSON.

6.5 Architecture du front

Le dossier src/ est découpé en api/ (un fichier par ressource et un client commun), contexts/ et hooks/ (authentification), pages/, components/ et utils/ (permissions, constantes).

7. Base de données

7.1 Modèle

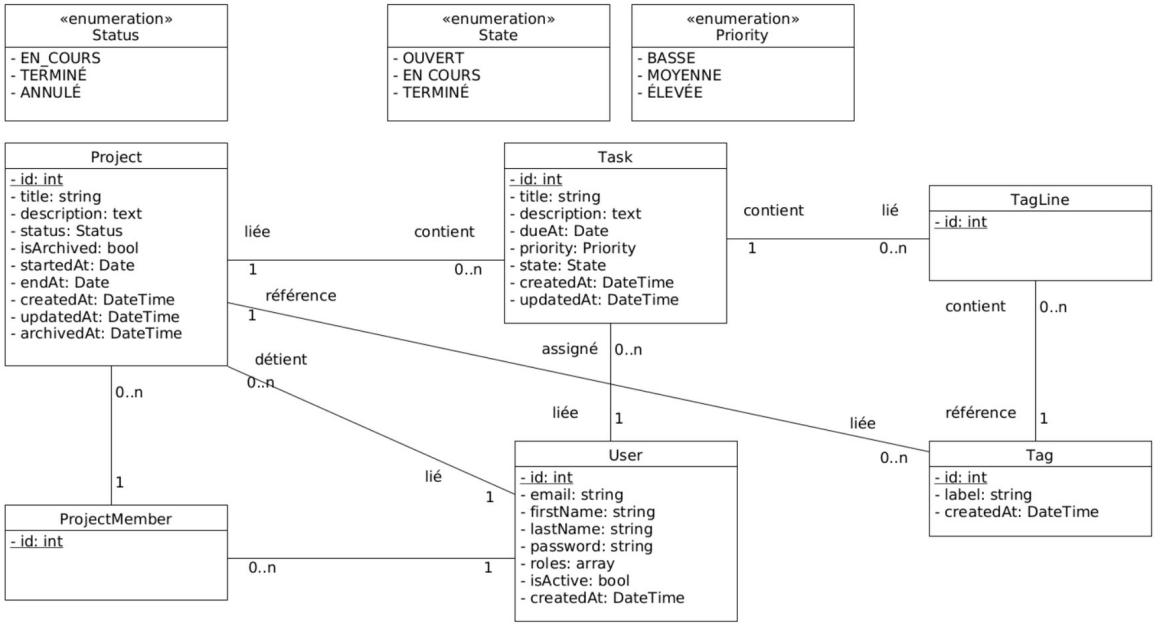


Diagramme de classes

Table	**Contenu**	**Clés**
user	email, mot de passe hashé, prénom, nom, rôles, actif	PK id, email unique
project	titre, description, statut, archivé, dates	PK id, FK owner_id
project_member	membres d'un projet	PK (project_id, user_id)
task	titre, description, échéance, priorité, état	PK id, FK project_id, FK assignee_id (nullable)
tag	libellé	PK id, FK project_id
task_tag	tags d'une tâche	PK (task_id, tag_id)

7.2 Mise en place

Les entités sont mappées avec les attributs Doctrine (#[ORM\Column], #[ORM\ManyToOne], #[ORM\ManyToMany]…) et validées avec les contraintes Symfony (#[Assert\NotBlank], #[Assert\Length]…). Le schéma évolue uniquement par **migrations Doctrine**. Des **fixtures** créent des comptes de démonstration pour chaque rôle.

```php
#[ORM\Entity(repositoryClass: ProjectRepository::class)]
#[ORM\HasLifecycleCallbacks]
class Project
{
    1 usage
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    2 usages
    #[Assert\NotBlank(message: 'Le titre est obligatoire.')]
    #[Assert\Length(max: 100)]
    #[ORM\Column(length: 100)]
    private ?string $title = null;

    2 usages
    #[ORM\Column(type: Types::TEXT, nullable: true)]
    private ?string $description = null;

    5 usages
    #[ORM\Column(enumType: ProjectStatus::class)]
    private ProjectStatus $status = ProjectStatus::IN_PROGRESS;

    6 usages
    #[ORM\ManyToOne(inversedBy: 'projects')]
    #[ORM\JoinColumn(nullable: false)]
    private ?User $owner = null;
```

Extrait de l'entité Project

8. Réalisations back-end

8.1 Les règles métier

Les droits sont écrits une seule fois, dans les entités, et réutilisés par tous les contrôleurs.

Un projet archivé n'est donc modifiable par personne. Pour les tâches, l'utilisateur assigné peut changer l'état de sa tâche, mais pas vers « en cours » :

```php
public function canChangeStateBy(User $user, TaskState $targetState): bool
{
    $project = $this->project;
    if (!$project instanceof Project || !$project->isEditable()) {
        return false;
    }

    if ($user->isManager()) {
        return true;
    }

    if ($project->isOwner($user)) {
        return true;
    }

    $assignee = $this->assignee;
    if ($assignee instanceof User && $assignee->getId() === $user->getId()) {
        return TaskState::IN_PROGRESS !== $targetState;
    }

    return false;
}
```

Méthode canChangeStateBy() de l'entité Task

Passer un projet en « terminé » ou « annulé » l'archive automatiquement ; le remettre « en cours » le restaure :

```php
public function restore(): void
{
    if (!$this->isArchived()) {
        throw new \LogicException( message: 'Impossible de restaurer un projet non archivé.');
    }

    $this->changeStatus( status: ProjectStatus::IN_PROGRESS);
}

3 usages  David
public function changeStatus(ProjectStatus $status): void
{
    if (ProjectStatus::COMPLETED === $status || ProjectStatus::CANCELLED === $status) {
        $this->status = $status;
        $this->isArchived = true;
        $this->setArchivedAt(new \DateTimeImmutable());

        return;
    }

    $this->status = ProjectStatus::IN_PROGRESS;
    $this->isArchived = false;
    $this->setArchivedAt( archivedAt: null);
}
```

Méthodes restore() et changeStatus() de l'entité Project

8.2 L'accès aux données

Les requêtes sont dans des repositories, avec le QueryBuilder de Doctrine. Les valeurs passent toujours par des **paramètres liés** ; le tri, qui ne peut pas être paramétré, est limité par une **liste blanche**.

```php
public function searchByProjectId(int $projectId, SearchTaskRequest $criteria): array
{
    $query = $this->createQueryBuilder( alias: 'task')
        ->andWhere( ...where: 'IDENTITY(task.project) = :projectId')
        ->setParameter( key: 'projectId', $projectId);

    if (!empty($criteria->state)) {
        $query->andWhere( ...where: 'task.state IN (:states)')
            ->setParameter( key: 'states', $criteria->state);
    }

    if (!empty($criteria->priority)) {
        $query->andWhere( ...where: 'task.priority IN (:priorities)')
            ->setParameter( key: 'priorities', $criteria->priority);
    }

    if (null !== $criteria->dueBefore) {
        $query->andWhere( ...where: 'task.dueAt IS NOT NULL')
            ->andWhere( ...where: 'task.dueAt <= :dueBefore')
            ->setParameter(
                key: 'dueBefore',
                new \DateTimeImmutable( datetime: $criteria->dueBefore.' 23:59:59'),
            );
    }

    if (!empty($criteria->tags)) {
        $query->innerJoin( join: 'task.tags', alias: 'tag')
            ->andWhere( ...where: 'tag.id IN (:tagIds)')
            ->setParameter( key: 'tagIds', $criteria->tags)
            ->distinct();
    }
```

Recherche multicritère des tâches (TaskRepository::searchByProjectId)

8.3 La sécurité de l'API

Deux pare-feux : login vérifie l'email et le mot de passe puis génère le token ; api exige un token valide sur toutes les autres routes. Seules /auth/login et /api/doc sont publiques.

```yaml
firewalls:
    dev:
        pattern: ^/(_(profiler|wdt)|css|images|js)/
        security: false

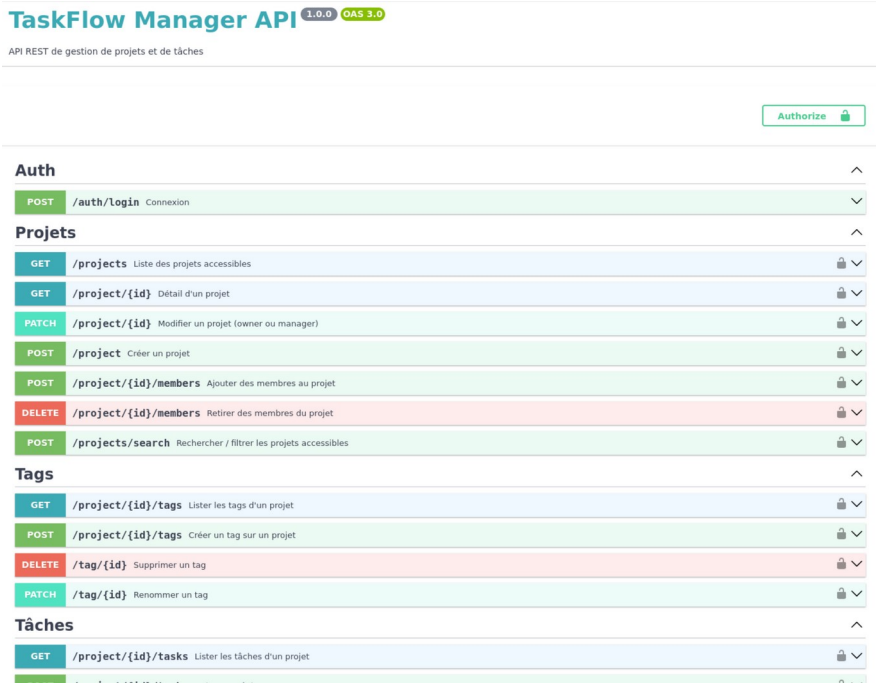
    login:
        pattern: ^/auth/login
        stateless: true
        provider: app_user_provider
        user_checker: App\Security\UserChecker
        json_login:
            check_path: /auth/login
            username_path: email
            password_path: password
            success_handler: lexik_jwt_authentication.handler.authentication_success
            failure_handler: lexik_jwt_authentication.handler.authentication_failure

    api:
        pattern: ^/
        stateless: true
        provider: app_user_provider
        user_checker: App\Security\UserChecker
        jwt: ~

access_control:
    - { path: ^/auth/login, roles: PUBLIC_ACCESS }
    - { path: ^/api/doc, roles: PUBLIC_ACCESS }
    - { path: ^/, roles: IS_AUTHENTICATED_FULLY }
```

Configuration security.yaml

Chaque route est documentée avec des attributs OpenAPI ; la documentation Swagger est disponible sur /api/doc.



Documentation Swagger de l'API

9. Réalisations front-end

9.1 Le client HTTP

Toutes les requêtes passent par une seule fonction, qui ajoute le token et gère la session expirée.

9.2 Les routes protégées

ProtectedRoute renvoie vers la connexion si l'utilisateur n'est pas connecté ; ManagerRoute réserve la page Utilisateurs aux managers.

```jsx
function App() {
  return (
    <Routes>
        <Route path="/login" element={<LoginPage />} />

        <Route element={<ProtectedRoute />}>
            <Route element={<DashboardLayout />}>
                <Route path="/projects" element={<ProjectsPage />} />
                <Route path="/projects/create" element={<ProjectCreatePage />} />
                <Route path="/projects/:id" element={<ProjectDetailPage />} />
                <Route path="/projects/:id/edit" element={<ProjectEditPage />} />
                <Route path="/profile" element={<ProfilePage />} />
                <Route element={<ManagerRoute />}>
                    <Route path="/users" element={<UsersPage />} />
                </Route>
            </Route>
        </Route>

        <Route path="*" element={<Navigate to="/projects" />} />
    </Routes>
  )
}
```

Routes de l'application (App.jsx)

9.3 Une interface qui s'adapte aux droits

Les fonctions de permissions.js reprennent la matrice d'autorisation et affichent ou masquent les boutons. Elles servent au confort de l'utilisateur : la vraie sécurité reste dans l'API.

```jsx
export function canEditProject(user, project) {
    if (!user || !project) {
        return false;
    }

    if (project.isArchived) {
        return false;
    }

    return isManager(user) || isOwner(user, project);
}

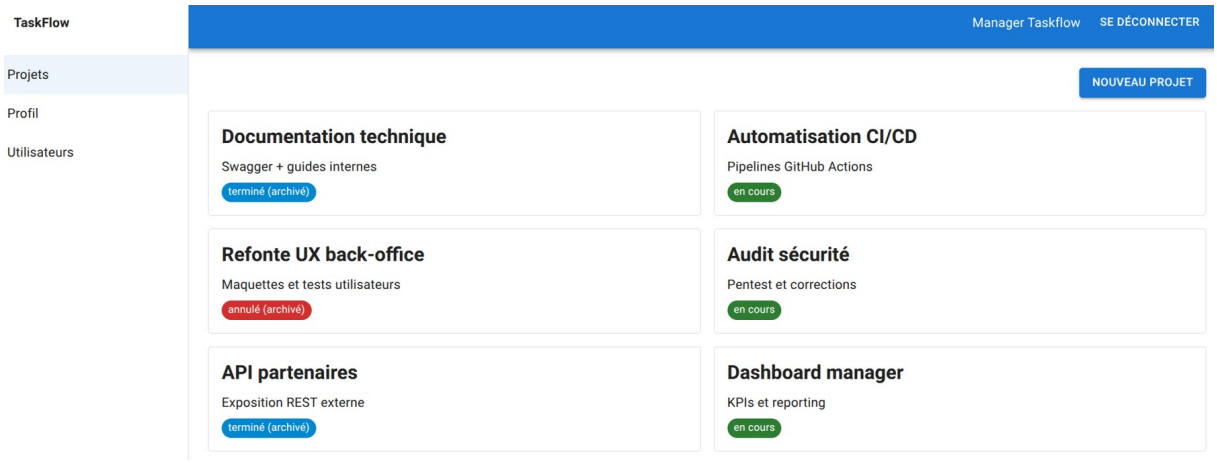
Show usages  David
export function canManageProject(user, project) {
    if (!user || !project) {
        return false;
    }

    return isManager(user) || isOwner(user, project);
}

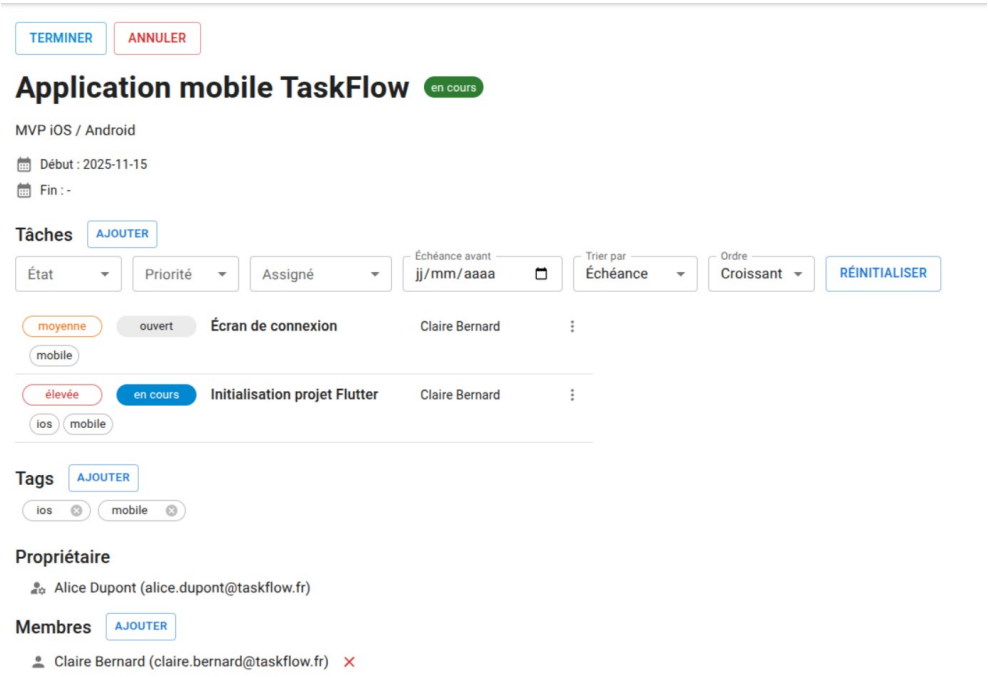
Show usages  David
export function canArchiveProject(user, project) {
    return canManageProject(user, project) && !project.isArchived;
}
```

Extrait de permissions.js

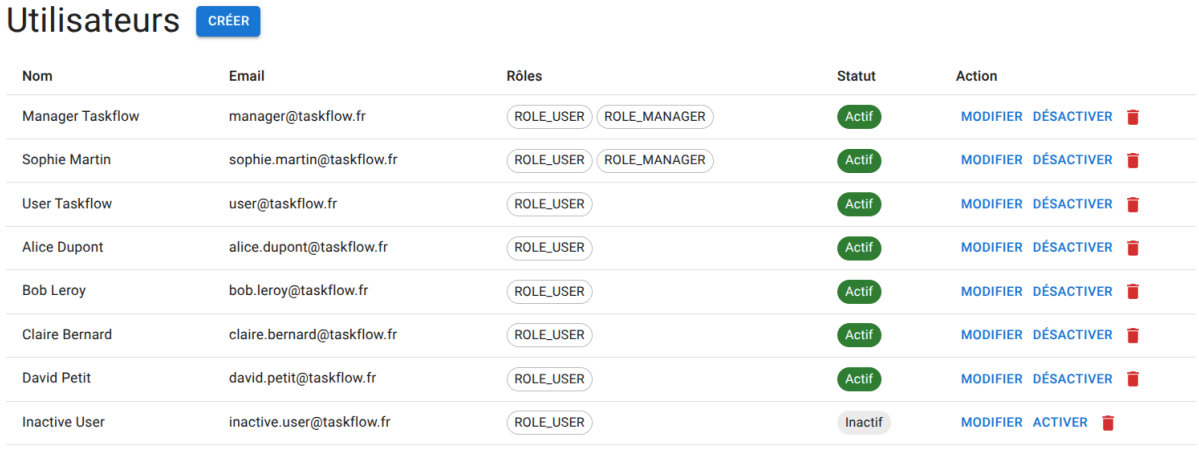
9.4 Les écrans



Liste des projets



Détail d'un projet : tâches, filtres, tags et membres



Administration des utilisateurs (manager)

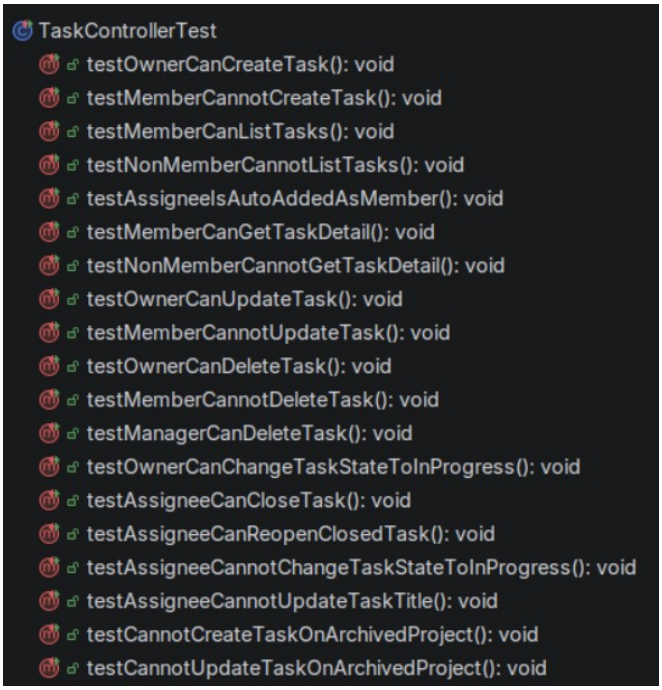
10. Sécurité

Risque	Protection mise en place
Accès non autorisé	Règles métier vérifiées par l'API à chaque requête, d'après la matrice d'autorisation
Injection SQL	Doctrine et requêtes préparées (paramètres liés)
Mot de passe	Mots de passe hashés (algorithme auto de Symfony), jamais renvoyés par l'API
Session	Token JWT signé, à durée limitée ; compte désactivé refusé (UserChecker)
Données invalides	DTO + Validator : seuls les champs prévus sont acceptés et contrôlés
XSS	React échappe l'affichage par défaut ; aucun HTML injecté
CSRF	API sans cookie de session : le token est envoyé dans un header
Appels depuis un autre site	CORS limité à l'URL du front
Secrets	Secrets en variables d'environnement, fichiers .env.local / .env.docker ignorés par Git

11. Tests

11.1 Stratégie

- **API** : 110 tests fonctionnels PHPUnit. Ils envoient de vraies requêtes avec un vrai token JWT, sur une base SQLite en mémoire recréée à chaque test ;
- Pour chaque route : le cas autorisé, les refus, les données invalides, les conflits et le cas du projet archivé ;
- **Front** : 62 tests Vitest et Testing Library (permissions, routes protégées, formulaires, pages) ;
- Les tests tournent avant chaque push (composer check, npm run test:run) et dans la CI.



Extrait de la liste des tests des tâches

11.2 Jeu de données : changement d'état d'une tâche

Route PATCH /task/{id} avec le corps {"state": "..."}. Projet en cours, tâche « ouvert » assignée à un utilisateur.

Utilisateur	Donnée envoyée	Résultat attendu	Résultat obtenu
Owner	state = « en cours »	200, état « en cours »	Conforme
Assigné	state = « terminé »	200, état « terminé »	Conforme
Assigné	state = « en cours »	403 Accès refusé	Conforme
Assigné	title = « Hack »	403 Accès refusé	Conforme
Owner	state = état actuel	400 Transition d'état invalide	Conforme
Owner	state = « terminé » sur un projet archivé	403 Accès refusé	Conforme
Sans token	state = « terminé »	401	Conforme

```php
public function testAssigneeCannotChangeTaskStateToInProgress(): void
{
    ['projectId' => $projectId, 'token' => $ownerToken] = $this->createProjectAs();
    $taskId = $this->addAssignedTaskToProject($projectId, $ownerToken, assigneeEmail: self::EMAIL_ASSIGNEE, ['title' => 'Tâche assignée']);

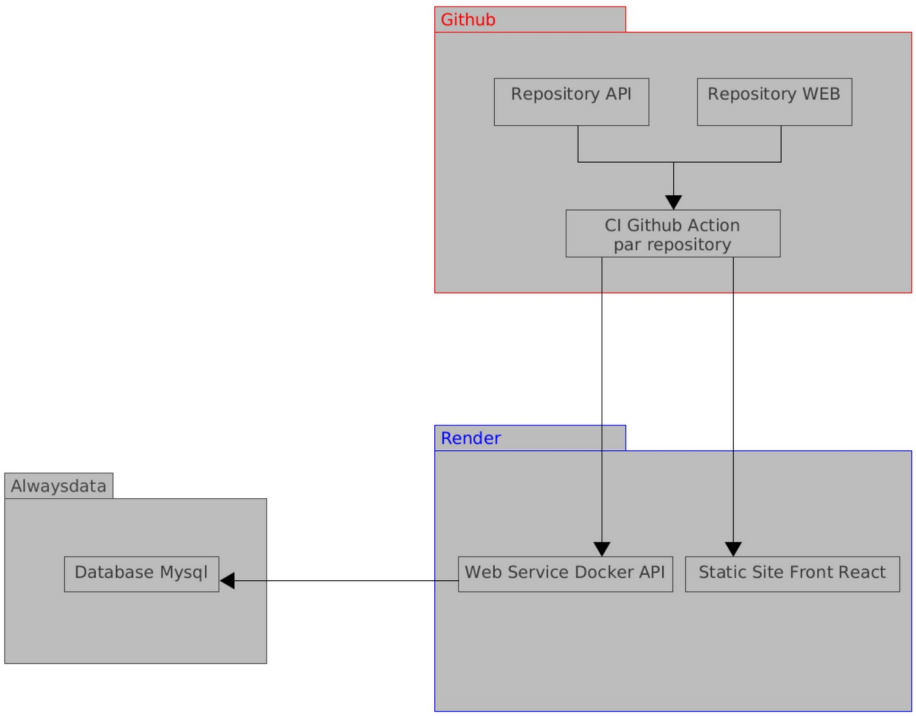
    $assigneeToken = $this->loginAndGetToken( email: self::EMAIL_ASSIGNEE, password: self::PASSWORD_USER);
    $this->updateTaskById($taskId, ['state' => TaskState::IN_PROGRESS->value], $assigneeToken);

    self::assertResponseStatusCodeSame( expectedCode: Response::HTTP_FORBIDDEN);
}
```

Test automatisé correspondant à la ligne 3

12. Déploiement et DevOps

12.1 Architecture de déploiement



Architecture de déploiement

12.2 Docker

- **API** : image php:8.2-apache, DocumentRoot sur public/, dépendances Composer, génération des clés JWT ;
- **Front** : image multi-stage, Node pour le build puis nginx pour servir les fichiers statiques ;
- compose.yaml lance l'API et MySQL (volume pour les données) ; la configuration passe par des variables d'environnement.

```dockerfile
FROM php:8.2-apache

RUN apt-get update && apt-get install -y libzip-dev \
    && docker-php-ext-install pdo_mysql zip \
    && a2enmod rewrite

ENV APACHE_DOCUMENT_ROOT=/var/www/html/public
RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available/*.conf \
    && sed -ri -e 's!/var/www/!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/apache2.conf /etc/apache2/conf-available/*.conf

WORKDIR /var/www/html
COPY . .

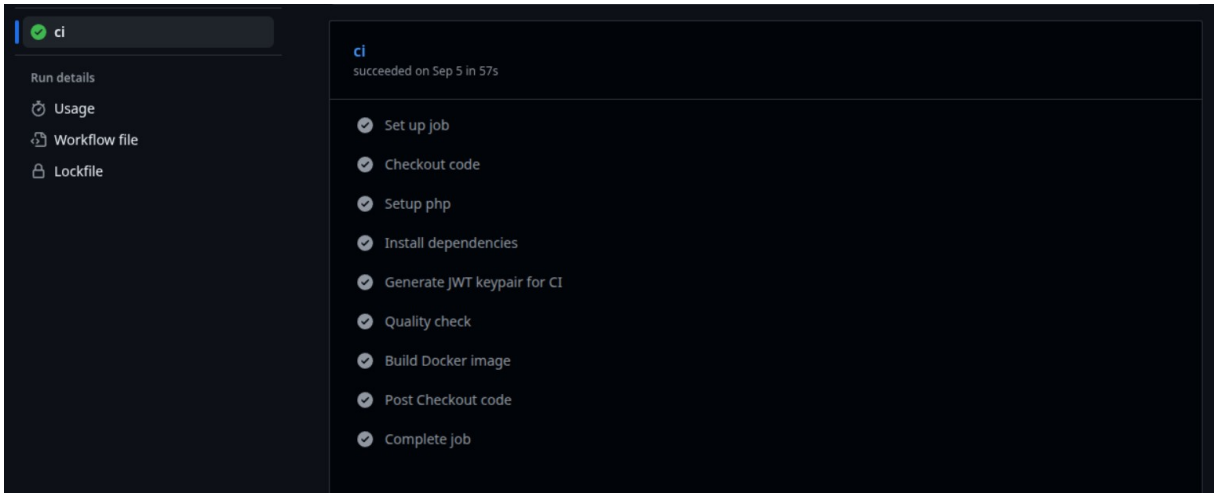
RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer \
    && composer install --optimize-autoloader \
    && chown -R www-data:www-data var

RUN mkdir -p config/jwt \
    && php bin/console lexik:jwt:generate-keypair --overwrite --no-interaction
```

Dockerfile de l'API

12.3 Intégration continue

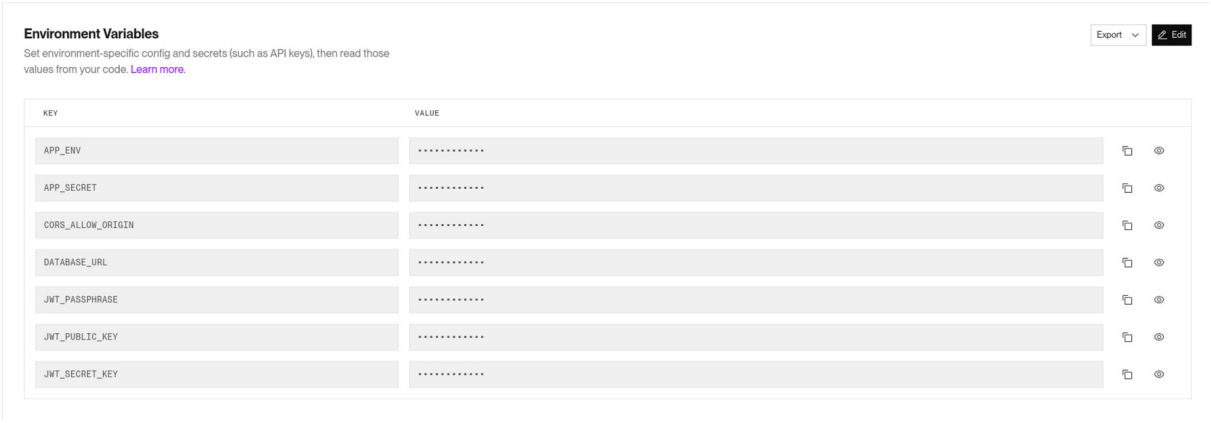
À chaque push, GitHub Actions installe le projet puis lance la qualité (PHPStan, PHPMD, PHP CS Fixer), les tests et la construction de l'image Docker. Côté front : lint, tests, build et image Docker. Une étape en échec bloque la fusion.



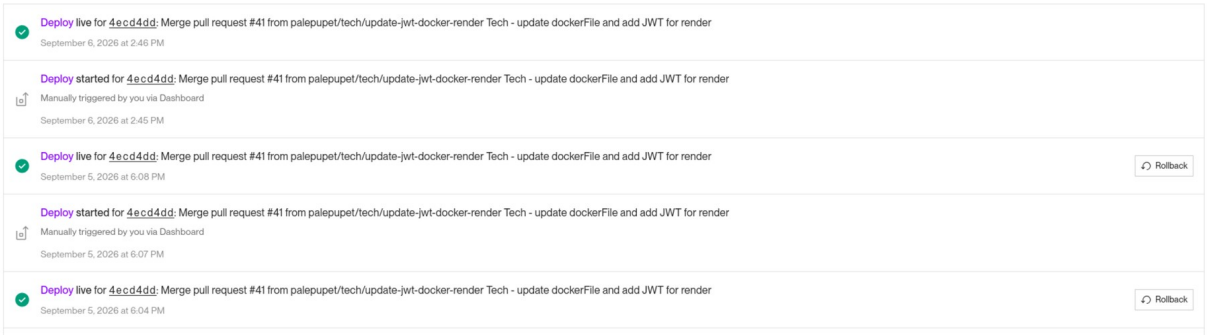
Pipeline CI de l'API

12.4 Mise en production

L'API est déployée sur Render (Web Service Docker), le front sur Render (site statique), la base MySQL chez Alwaysdata. Le déploiement se fait à chaque fusion sur `main`. Les secrets sont saisis dans Render. La production est en HTTPS, en `APP_ENV=prod`, avec un CORS limité au front.



Variables d'environnement sur Render



Historique des déploiements sur Render

13. Bilan

TaskFlow Manager est fonctionnel et en ligne. Le projet m'a permis de réaliser une application complète : analyse, conception, API sécurisée, interface web, tests, conteneurisation et mise en production.

14. Annexes

14.1 Cas d'utilisation par rôle

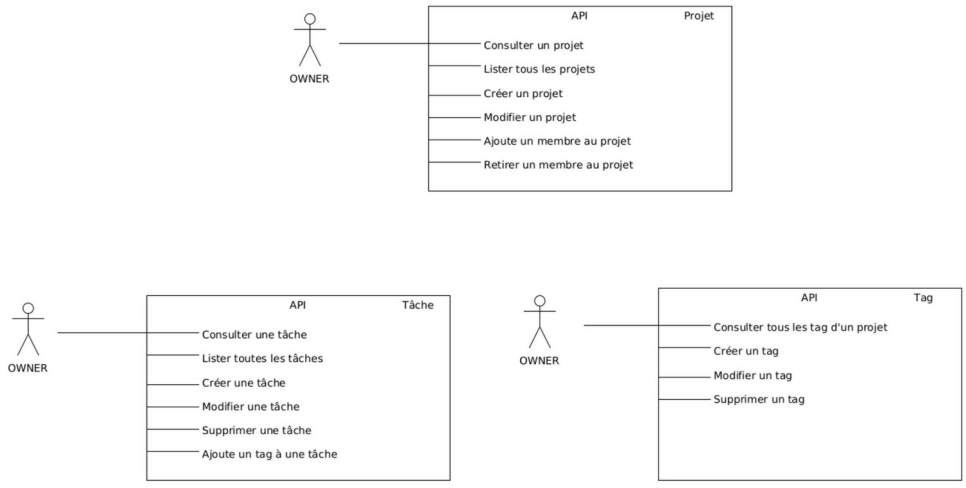


Diagramme de cas d'utilisation - Owner

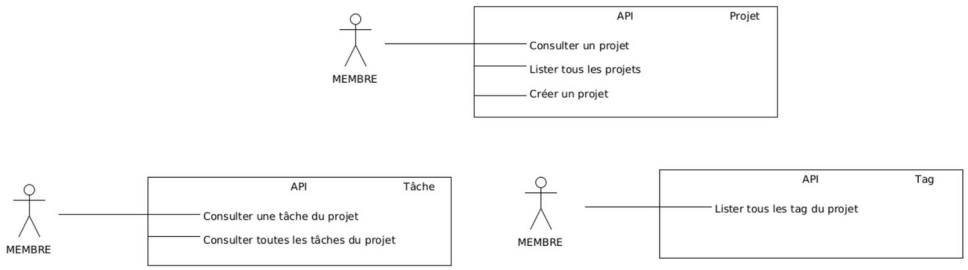


Diagramme de cas d'utilisation - Membre

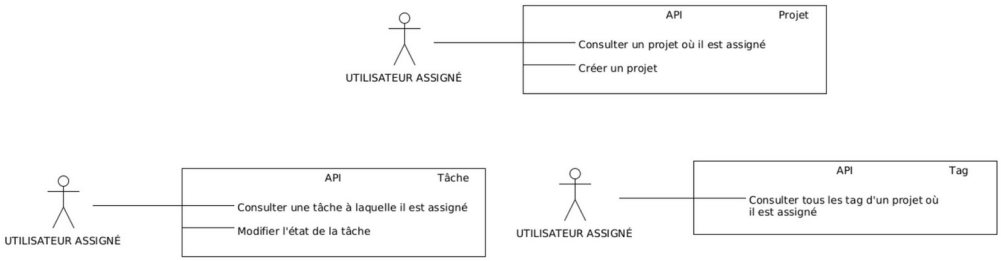


Diagramme de cas d'utilisation - Assigné

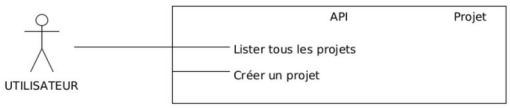
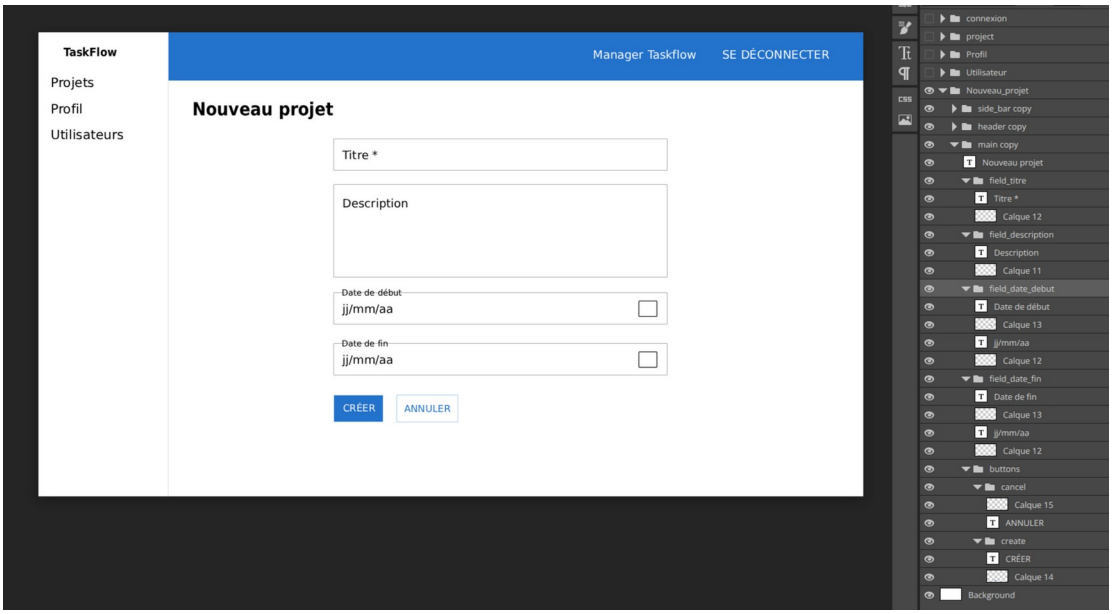
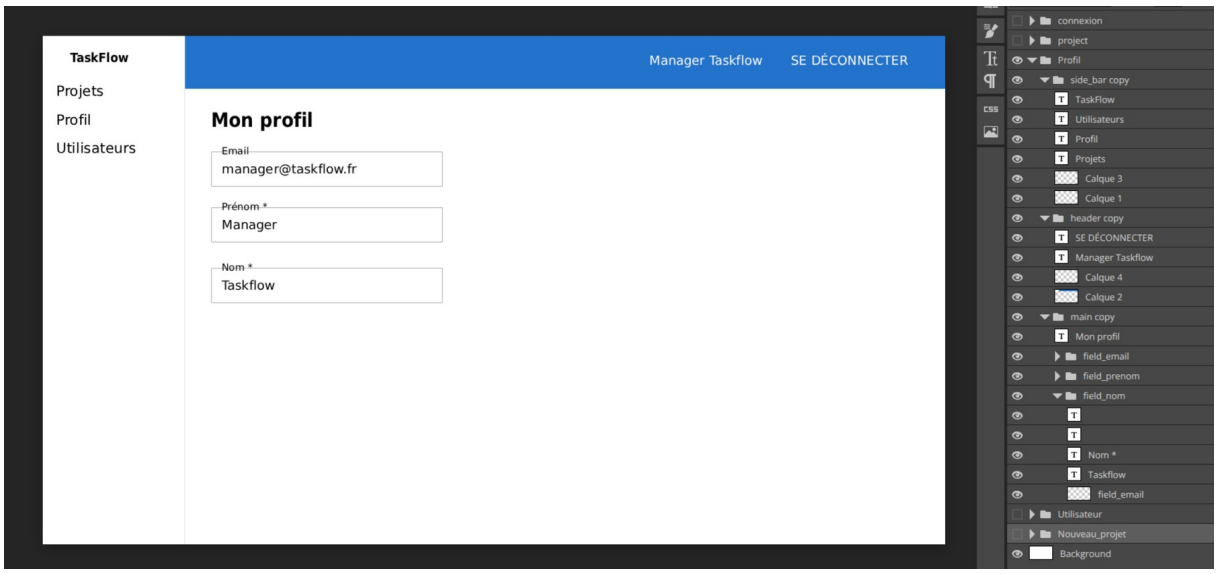


Diagramme de cas d'utilisation - Utilisateur

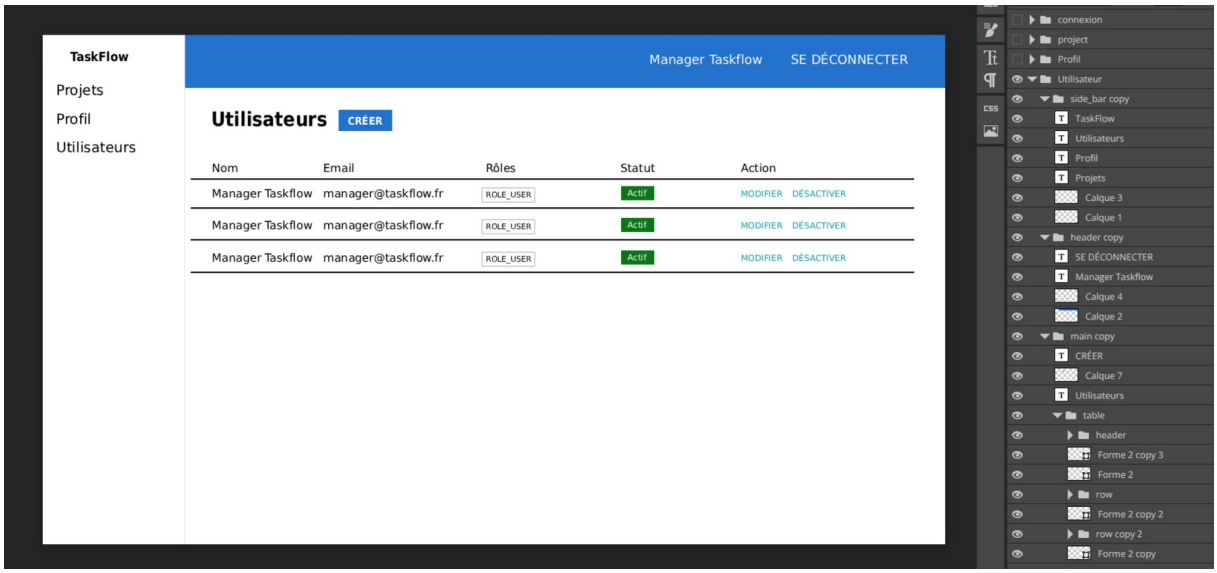
14.2 Autres maquettes



Maquette - Nouveau projet

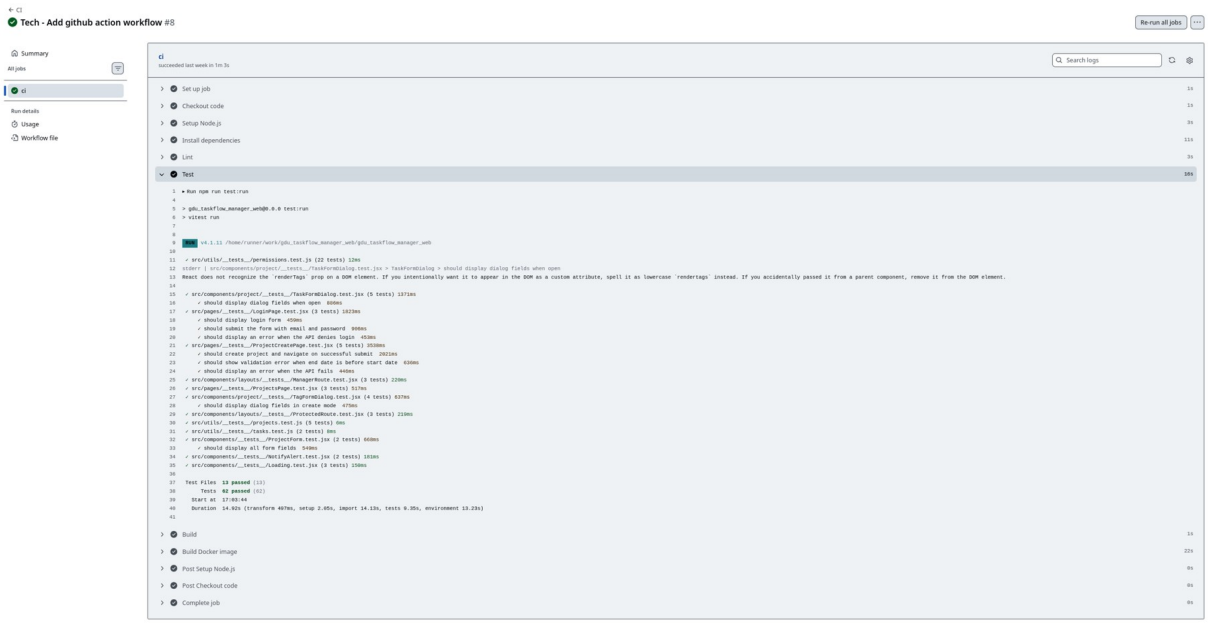


Maquette - Profil



Maquette - Utilisateurs

14.3 Pipeline CI du front



Pipeline CI du front avec le rapport des tests

14.4 Liens

- Dépôt de l'API : https://github.com/palepupet/gdu_taskflow_manager_api
- Dépôt du front : https://github.com/palepupet/gdu_taskflow_manager_web
- Application en ligne : https://gdu-taskflow-manager-web.onrender.com
- Documentation de l'API : https://gdu-taskflow-manager-api.onrender.com/api/doc
- Tableau Trello : https://trello.com/b/KJUtMTTP/task-flow-manager-api